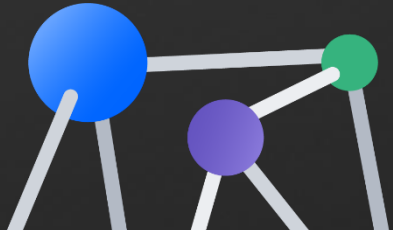


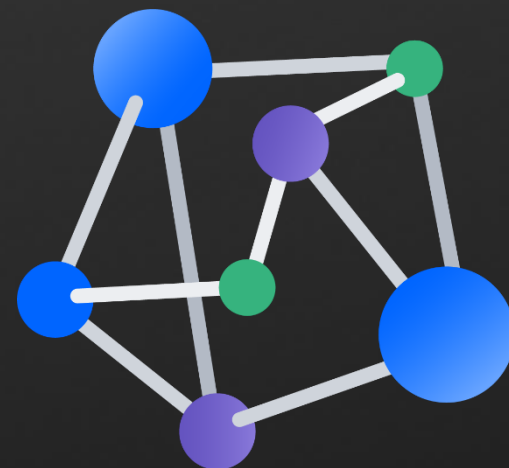
Git应用分享

目录

- Git简介
- Git使用
- 版本库目录规范
- 编码规范

关于版本控制

- 什么是“版本控制”？
 - 版本控制是一种记录一个或若干文件内容变化，以便将来查阅特定版本修订情况的系统。
 - 版本控制系统（VCS）带来的好处
 - 可以将选定的文件回溯到之前的状态，甚至将整个项目都回退到过去某个时间点的状态；
 - 可以比较文件的变化细节，查出最后是谁修改了哪个地方，从而找出导致问题的原因；
 - 就算你把整个项目中的文件改的改删的删，你也照样可以轻松恢复到原先的样子。
 - 版本控制系统分类
 - 本地版本控制系统（如：RCS）
 - 集中化的版本控制系统（如：CVS、Subversion）
 - 分布式版本控制系统（如：Git、Mercurial、Bazaar）
- 





Git简介

- Git（吉特）是一个分布式版本控制软件
 - 最初由林纳斯·托瓦兹（Linus Torvalds）创作。
 - 最初目的是为更好地管理Linux内核开发而设计。
- 命名来源：

源代码的自述文件进一步阐述了：

The name "git" was given by Linus Torvalds when he wrote the very first version. He described the tool as "the stupid content tracker" and the name as (depending on your way):

- random three-letter combination that is pronounceable, and not actually used by any common UNIX command. The fact that it is a mispronunciation of "get" may or may not be relevant.
- "global information tracker": you're in a good mood, and it actually works for you. Angels sing, and a light suddenly fills the room.
- stupid. contemptible and despicable. simple. Take your pick from the dictionary of slang.

林纳斯·托瓦兹在编写第一个版本时就使用了“git”这个名称。他将工具描述为“愚蠢的内容跟踪器”，并将其描述为（取决于您的方式）：

- 可以发音念出的随机三个字母组合，而且并未被实际用在任何 UNIX 指令上。它是“get”的错误发音，这点可能相关也可能无关。
- “全球信息跟踪器”：您的心情不错，对你而言它也确实说得通。天使唱歌，房间突然充满光明。
- 愚蠢的。鄙视和卑鄙的。简单。从俚语字典中选择。



Git 简史

- 1991-2002年间，绝大多数的Linux内核维护工作都花在了提交补丁和保存归档的繁琐事务上。
- 到了2002年，整个项目组开始启用BitKeeper来管理和维护代码。
- 到了2005年，开发BitKeeper的商业公司收回免费使用权，迫使Linux开源社区开发自己的版本系统。
- 版本系统订制的目标
 - 速度
 - 简单的设计
 - 对非线性开发模式的强力支持（允许成千上万个并行开发的分支）
 - 完全分布式
 - 有能力高效管理类似Linux内核一样的超大规模项目（速度和数据量）

Git的文件状态

- Git有三种状态。

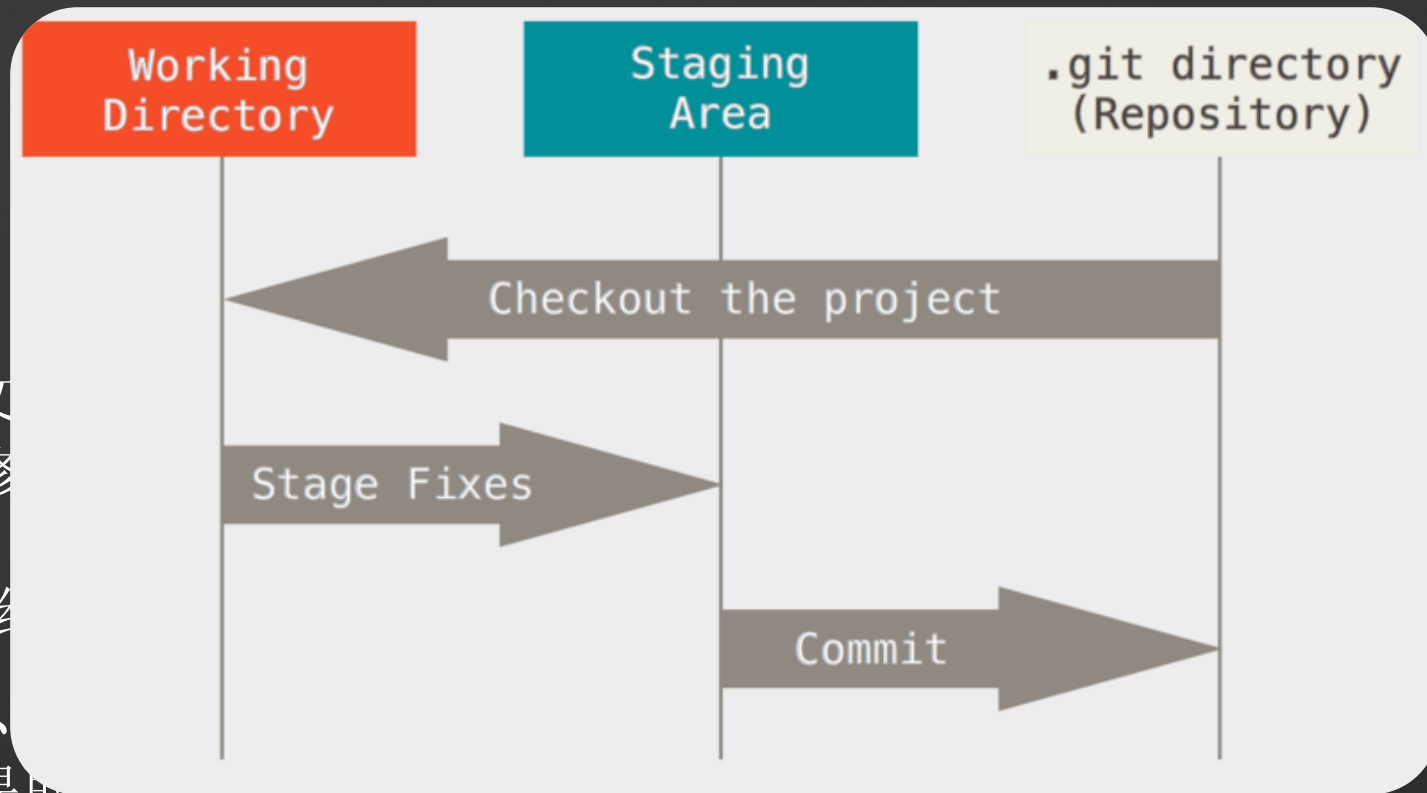
- 已修改 (modified) 表示修改了文件
- 已暂存 (staged) 表示对一个已修改文件的快照中。
- 已提交 (committed) 表示数据已经

- Git项目拥有三个阶段：工作区、

- **工作区**是对项目的某个版本独立提取出来的内容。
- **暂存区**是一个文件，保存了下次将要提交的文件列表信息。
- **Git目录**是Git用来保存项目的元数据和对象数据库的地方。

- 基本的Git工作流程如下：

- 1、在工作区中修改文件。
- 2、将你想要提交的更改选择性地暂存，这样只会将更改的部分添加到暂存区。
- 3、提交更新，找到暂存区的文件，将快照永久性存储到Git目录。



Git环境设置

- Git的安装，我这里已经专门录制了视频，这里强调首次配置。
 - 下载地址：
 - <https://pan.baidu.com/s/1bZ25NDZuk-1c6b96aRa1Aw?pwd=sse9>
 - 提示：下载地址不要复制打开，直接 *ctrl+鼠标左键* 点击打开链接
 - 提取码：sse9

用户信息

安装完 Git 之后，要做的第一件事就是设置你的用户名和邮件地址。这一点很重要，因为每一个 Git 提交都会使用这些信息，它们会写入到你的每一次提交中，不可更改：

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

再次强调，如果使用了 `--global` 选项，那么该命令只需要运行一次，因为之后无论你在该系统上做任何事情，Git 都会使用那些信息。当你想针对特定项目使用不同的用户名称与邮件地址时，可以在那个项目目录下运行没有 `--global` 选项的命令来配置。

很多 GUI 工具都会第一次运行时帮助你配置这些信息。

检查配置信息

如果想要检查你的配置，可以使用 `git config --list` 命令来列出所有 Git 当时能找到的配置。

```
$ git config --list
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
...
```

你可能会看到重复的变量名，因为 Git 会从不同的文件中读取同一个配置（例如：`/etc/gitconfig` 与 `~/.gitconfig`）。这种情况下，Git 会使用它找到的每一个变量的最后一个配置。

你可以通过输入 `git config <key>`：来检查 Git 的某一项配置

```
$ git config user.name
John Doe
```

Git常用命令-本地操作

- `git init`: 初始化工作目录
- `git add`: 添加文件, 如`git add *.txt`
- `git status`: 跟踪文件状态
- `git rm`: 从版本库移除文件
- `git mv`: 移动文件
- `git commit`: 提交文件, 如`git commit -m 日志信息`
- `git log`: 查看提交日志
- `git reset`: 取消暂存的文件
- `git checkout`: 撤消对文件的修改

Git常用命令-远程操作

- `git remote [-v]`: 查看你已经配置的远程仓库服务器
- `git remote add <lname> <url>`: 添加远程仓库
- `git remote rename <lname> <newname>`: 远程仓库重命名
- `git remote remove <lname>`: 移除远程仓库
- `git remote show <lname>`: 查看某个远程仓库
- `git fetch <lname>`: 从远程仓库中抓取, 不影响工作目录
- `git pull <lname> <branch>`: 从远程仓库中拉取到工作目录
 - `--allow-unrelated-histories`:
- `git push <lname> <branch>`: 推送到远程仓库
- `git clone <url>`: 克隆项目

Git 标签

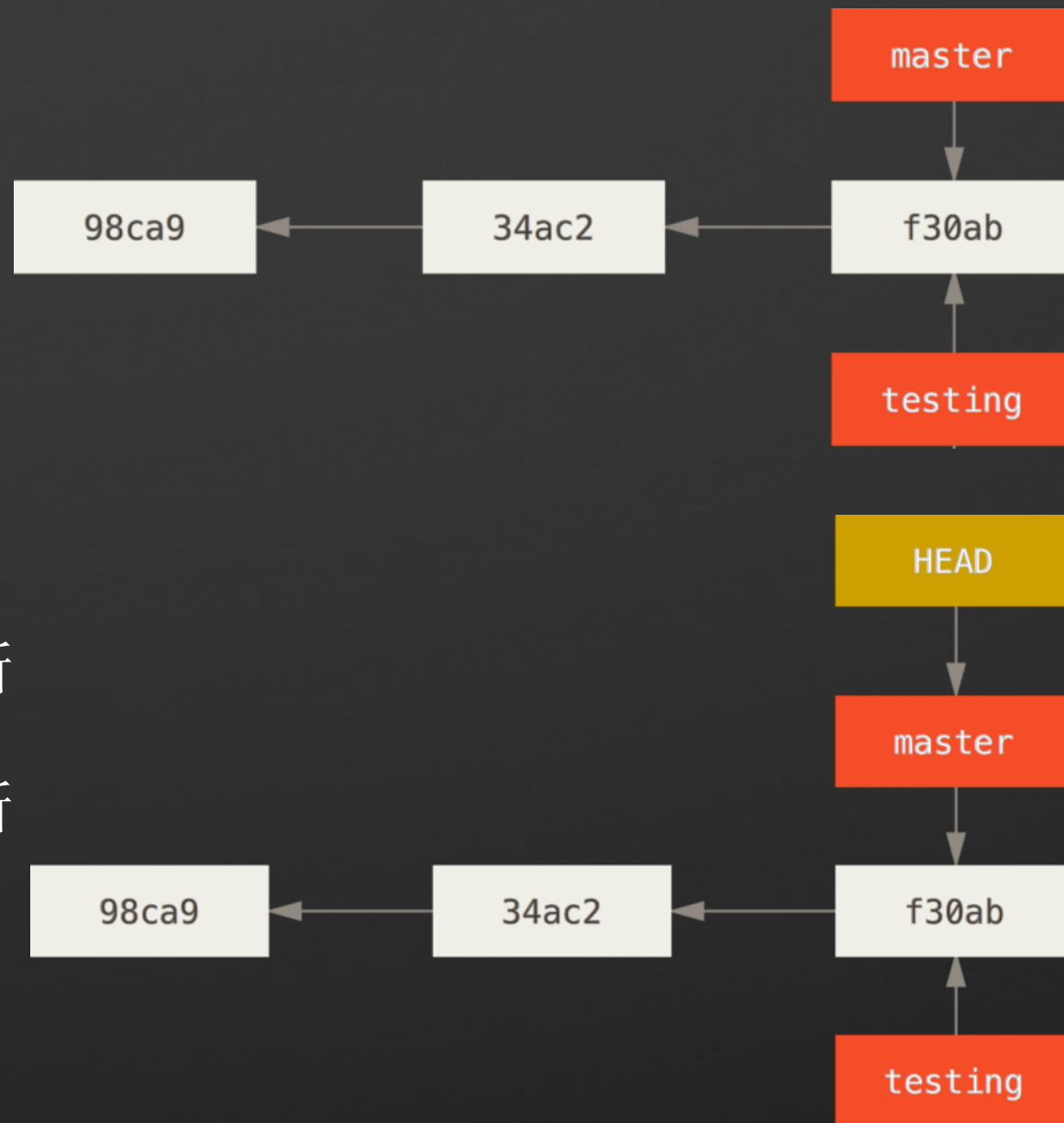
- 使用标签功能来标记发布结点（v1.0 、 v2.0）
 - `git tag`
- 标签分类
 - 轻量标签（lightweight）：本质上是将提交校验和存储到一个文件中，没有保存任何其他信息；
 - `git tag <标签名>`
 - 附注标签（annotated）：是存储在Git数据库中的一个完整对象，它们是可以被校验的。
 - `git tag -a <标签名> -m <附注描述信息>`
- 共享标签
 - 默认情况下，`git push`命令并不会传送标签到远程仓库服务器上。
 - 推送一个标签：`git push origin <标签名称>`
 - 推送所有标签：`git push origin --tags`

Git分支概念

- 使用分支意味着你可以把你的工作从开发主线上分离开来，以免影响开发主线。
- 在很多版本控制系统中，完全创建一个源代码目录的副本。
- Git处理分支的方式可谓是难以置信的轻量，创建新分支几乎能在瞬间完成，并且在不同分支之间的切换操作也是一样便捷。
- Git鼓励在工作流程中频繁地使用分支与合并，哪怕一天之内进行许多次。
- Git的默认分支名字是master。在多次提交操作之后，你其实已经有一个指向最后那个提交对象的master分支。

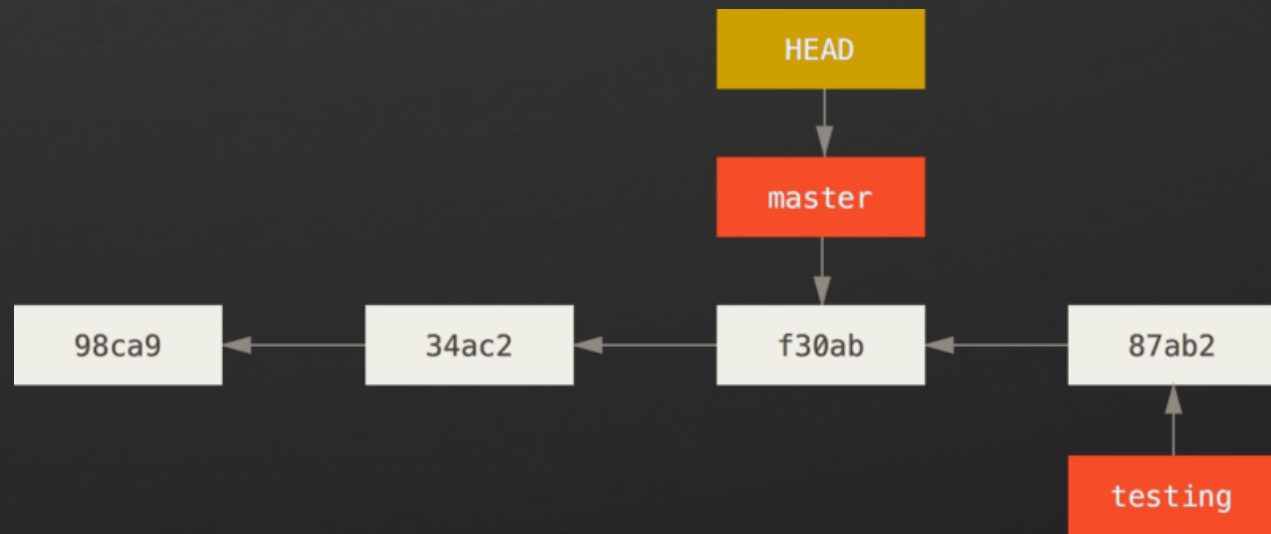
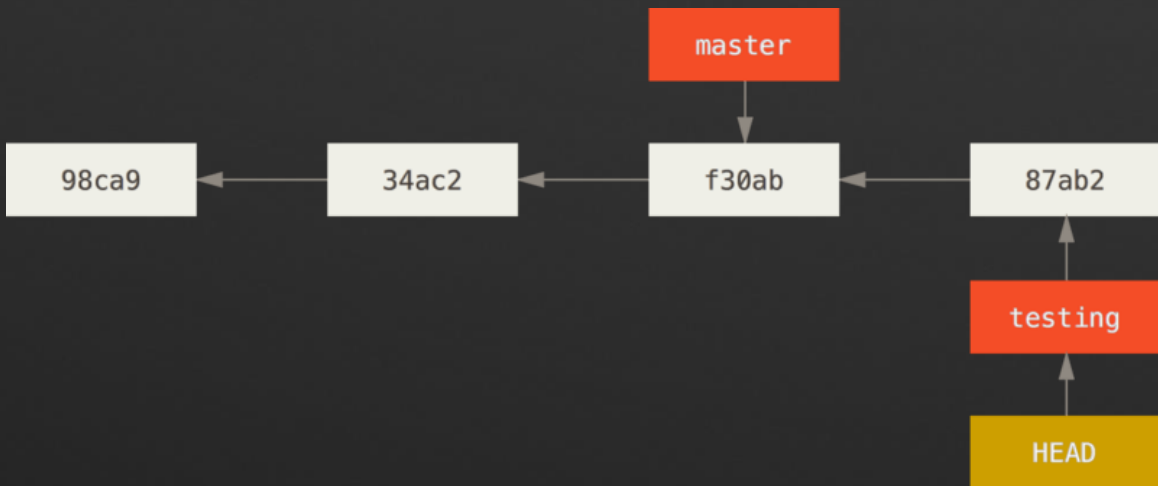
Git分支命令

- 创建分支
 - `git branch <name>`
- Git怎么知道当前在哪个分支上呢？
 - 它有一个名为HEAD的特殊指针, 向当前所在的本地分支
 - 使用 `git log` 命令查看各个分支当前所指的对象
 - `git log --oneline --decorate`



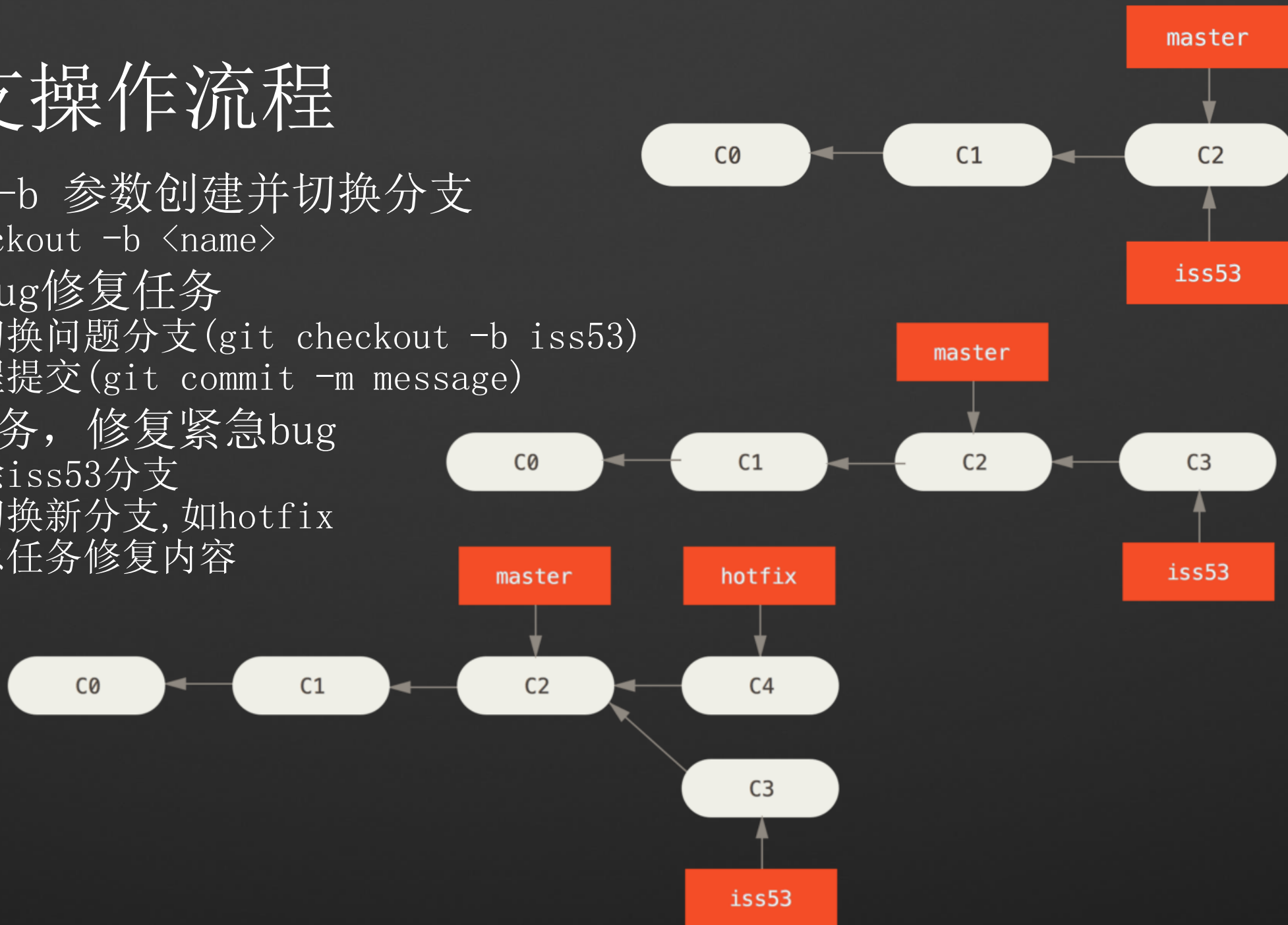
Git分支命令

- 分支切换
 - `git checkout <name>`
- 分支提交
- 分支切换回master



Git分支操作流程

- 可以使用 `-b` 参数创建并切换分支
 - `git checkout -b <name>`
- 接到一个bug修复任务
 - 创建并切换问题分支(`git checkout -b iss53`)
 - 修复过程提交(`git commit -m message`)
- 暂停bug任务, 修复紧急bug
 - 无需删除iss53分支
 - 创建并切换新分支, 如hotfix
 - 提交紧急任务修复内容



Git分支合并

- 首先切换到合并分支，如主分支
 - git checkout master
- 执行分支合并命令
 - git merge <name>
- 合并冲突解决
 - git status: 查看冲突
 - 冲突文件内容
 - git mergetool: 启用冲突解决工具

```
Administrator@... MINGW64 ~/Desktop/test (master)
$ git merge issu5
Auto-merging a.m
CONFLICT (content): Merge conflict in a.m
Automatic merge failed; fix conflicts and then commit the result.

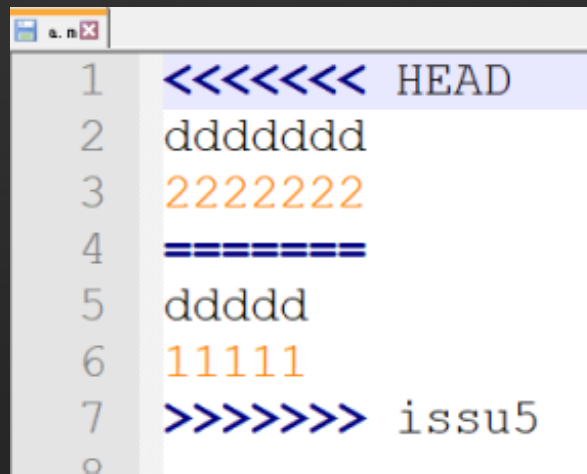
Administrator@... MINGW64 ~/Desktop/test (master|MERGING)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 2 commits.
(use "git push" to publish your local commits)

You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)

        both modified:   a.m

no changes added to commit (use "git add" and/or "git commit -a")
```



```
1 <<<<<< HEAD
2 dddddddd
3 2222222
4 =====
5 ddddd
6 11111
7 >>>>>> issu5
8
```

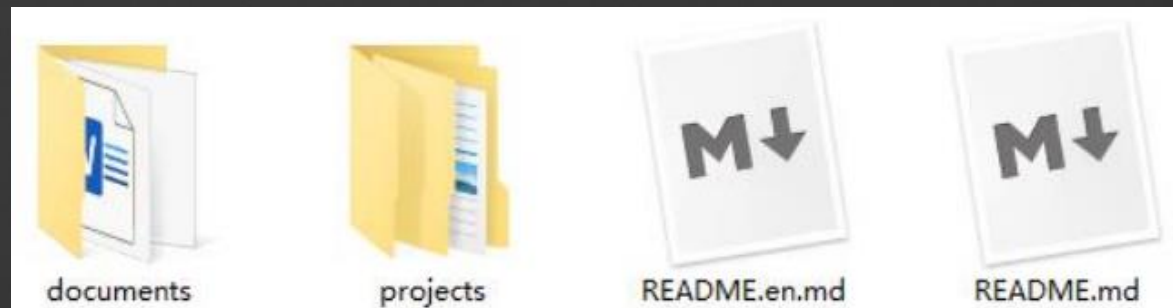
分支管理命令

- `git branch -v`: 查看每一个分支的最后一次提交。
- `git branch --merged`: 查看已合并的分支
- `git branch --no-merged`: 查看未合并的分支
- `git branch -d <name>`: 删除分支
 - 如果未合并的分支将会删除失败可以使用 `-D` 选项来强制删除

版本库目录规范

- 根目录结构

- documents: 项目相关文档文件夹
- projects: 项目代码文件夹
- README.md: 自读文件
- README.en.md: 自读文件（英文）
- 模板参考链接: <https://gitee.com/zero-awei/hello-gitee.git>



- 这里是项目名称
 - 介绍
 - 目录结构说明
 - 软件架构
 - 后端技术
 - 前端技术
 - 环境搭建
 - 开发工具
 - 开发环境
 - 安装教程
 - 开发进度

这里是项目名称

介绍

这里是项目介绍

目录结构说明

project-name

documents -- 项目开发相关文档（如：项目需求文档、设计文档等等）

projects -- 项目源码（比如：前端项目、后端项目等等）

编码规范

- 编码规范

- 阿里巴巴Java开发手册
- Google开源项目风格指南

- 编码检查工具

- Java开发规约IDE插件
 - 使用帮助:
 - <https://github.com/alibaba/p3c>

- Cpplint

- 使用帮助:
 - <https://github.com/zh-google-styleguide/zh-google-styleguide>
 - <https://cloud.tencent.com/developer/article/1494003>

